# Лабораторна робота №1. Налаштування локального середовища розробки: Node.js, Vite, ESLint, Prettier

> **Примітка щодо актуальності.** Документ підготовлено станом на 4 листопада 2026 року: Node.js 26 LTS, Vite 8, React 19, ESLint 10, Prettier 3. Версії, запитання `create-vite` і назви меню можуть змінитися; у разі відмінностей орієнтуйтеся на офіційну документацію (§8).

> **Якщо ви вже виконали ЛР-1 в окремому репозиторії `react-lab-1`,** не починайте заново: перенесіть проєкт у репозиторій для всіх лабораторних за §4.9.

## 1. Мета роботи

Підготувати локальне середовище React-розробки для наступних лабораторних: Node.js, репозиторій для всіх лабораторних курсу, проєкт на Vite, ESLint і Prettier, налаштований редактор, React Developer Tools, AI-агент, який знає правила проєкту з `AGENTS.md`, і деплой на Vercel.

**Очікуваний результат:** React-застосунок на Vite запускається командою `npm run dev`; `npm run lint`, `npm run format:check` і `npm run build` проходять без помилок; редактор форматує файл під час збереження й підкреслює помилки ESLint; AI-агент читає `AGENTS.md` і запускає перевірки, перш ніж звітувати; код — у теці `lab-1/` репозиторію `react-labs` на GitHub, а застосунок відкривається на Vercel.

## 2. Цілі роботи

- встановити Node.js LTS і пояснити, навіщо фронтенд-проєкту Node.js, npm і `package.json`;
- створити React-проєкт шаблоном Vite й розібратися в його структурі та скриптах;
- побачити, як працюють dev-сервер з HMR (React Fast Refresh) і production-збірка;
- налаштувати ESLint із правилами хуків React і Prettier так, щоб вони не конфліктували;
- налаштувати редактор: форматування під час збереження, підказки ESLint, спільні налаштування в репозиторії;

- встановити React Developer Tools і знайти в них компонент та його стан;
- написати `AGENTS.md` для React-проєкту й перевірити, що агент його дотримується;
- створити репозиторій для всіх лабораторних курсу, опублікувати його на GitHub і розгорнути проєкт на Vercel.

# 3. Завдання лабораторної роботи

1. Встановити Node.js LTS, Git, редактор і React Developer Tools (§4.1).
2. Створити репозиторій `react-labs`, у ньому — проєкт `lab-1` шаблоном Vite «React + JavaScript» з ESLint, і зробити перший коміт (§4.2).
3. Запустити dev-сервер і production-збірку, розібрати структуру проєкту (§4.3).
4. Додати Prettier і `eslint-config-prettier`, скрипти `format` і `format:check` (§4.4).
5. Налаштувати редактор (§4.5).
6. Перевірити інструменти на навмисних помилках (§4.6).
7. Написати `AGENTS.md` і перевірити агента двома запитами (§4.7).
8. Опублікувати репозиторій на GitHub, розгорнути застосунок на Vercel (§4.8) і надіслати звіт (§5).

# 4. Хід виконання роботи

## 4.1. Підготовка середовища

1. **Node.js.** Потрібен Node.js 22.13 або новіший. ESLint 10 працює на Node.js 20.19+, 22.13+ або 24+, але підтримку Node.js 20 завершено у квітні 2026 року. Рекомендовано актуальну LTS-версію — Node.js 26 (LTS з 28 жовтня 2026 року); Node.js 24 з курсу «HTML, CSS, JS/TS» теж підходить. Перевірте версії:

   ```
   node --version
   npm --version
   git --version
   ```

   Якщо проєкти потребують різних версій Node.js, зручно користуватися менеджером версій: fnm, nvm або Volta.
2. **Редактор.** Підійде редактор з курсу «HTML, CSS, JS/TS»: VS Code, Cursor, Windsurf або WebStorm. У VS Code, Cursor і Windsurf встановіть розширення **ESLint** (`dbaeumer.vscode-eslint`) і **Prettier - Code formatter** (`esbenp.prettier-vscode`). WebStorm підтримує обидва інструменти без розширень.
3. **Браузер.** Chrome або Edge з розширенням **React Developer Tools** (є також для Firefox). Після встановлення в DevTools з'являються вкладки ⚛ **Components** і ⚛

**Profiler** — на сторінках, зроблених на React.

4. **AI-агент.** Той самий, що в курсі «HTML, CSS, JS/TS», з режимом, у якому агент сам змінює файли (зазвичай Agent). Чи читає він `AGENTS.md`, — у таблиці §4.7.
5. **Vercel.** Акаунт з курсу «HTML, CSS, JS/TS», прив'язаний до GitHub. Якщо акаунта немає, створіть його на vercel.com через вхід з GitHub.

## 4.2. Репозиторій і проєкт

Усі лабораторні курсу виконуються в одному репозиторії `react-labs`, кожна — в окремій теці: `lab-1/`, `lab-2/` і так далі. Тека лабораторної — окремий застосунок Vite зі своїм `package.json` і налаштуваннями ESLint і Prettier. Файли, спільні для всіх лабораторних, лежать у корені репозиторію. Окремі репозиторії матимуть лише ЛР 20 (Next.js) і фінальний проєкт.

```
react-labs/
├── .gitattributes
├── .gitignore
├── .vscode/              налаштування редактора (§4.5)
├── AGENTS.md             правила для AI-агента (§4.7)
├── package.json          npm workspaces: перелік тек lab-*
├── package-lock.json     точні версії пакетів усіх лабораторних
├── node_modules/         пакети всіх лабораторних; не комітять
└── lab-1/
    ├── .gitignore
    ├── .prettierrc.json
    ├── eslint.config.js
    ├── index.html
    ├── package.json
    ├── vite.config.js
    ├── public/
    └── src/
```

1. У теці, де лежатимуть ваші проєкти, створіть репозиторій:

   ```
   mkdir react-labs
   cd react-labs
   git init -b main
   ```

2. У теці `react-labs` виконайте:

   ```
   npm create vite@latest
   ```

   Якщо npm запитає дозвіл установити `create-vite`, відповідайте `y`. Далі відповідайте так:

| Запитання | Відповідь |
|---|---|
| Project name | `lab-1` |
| Select a framework | React |
| Select a variant | JavaScript |
| Which linter to use? | ESLint |
| Install with npm and start now? | No |

**Чому так.** TypeScript з'явиться в ЛР 19, React Compiler — у ЛР 15, тож зараз — JavaScript без компілятора. За замовчуванням шаблон пропонує Oxlint — швидкий лінтер на Rust, який реалізує лише частину правил і плагінів ESLint. Курс використовує ESLint: правила хуків (`eslint-plugin-react-hooks`) розробляє сама команда React.

3. Створіть у корені репозиторію (у `react-labs/`, не в `lab-1/`) три файли.
   `package.json`:

```
{
  "name": "react-labs",
  "private": true,
  "workspaces": [
    "lab-*"
  ]
}
```

   Поле `workspaces` оголошує теки `lab-*` робочими просторами npm (npm workspaces). Команда `npm install` у корені встановлює пакети всіх лабораторних в одну `node_modules/` і записує їхні версії в один `package-lock.json` — без окремої копії на кожну лабораторну. Завдяки цьому Vercel також не перезбирає лабораторні, яких коміт не торкнувся (§4.8). `"private": true` не дає випадково опублікувати репозиторій у реєстрі npm.
   `.gitattributes`:

```
* text=auto eol=lf
```

   Git на Windows зазвичай перетворює кінці рядків на CRLF, коли записує файли в робочу теку. Prettier очікує LF, тож у свіжому клоні репозиторію (у викладача чи на іншому комп'ютері) `npm run format:check` позначив би кожен файл. Цей рядок змушує Git зберігати й видавати файли з LF.
   `.gitignore`:

```
node_modules
.idea
.DS_Store
```

`.gitignore` шаблону в `lab-1/` діє лише на файли цієї теки, а спільна `node_modules/` лежить у корені. `.idea/` — тека налаштувань WebStorm, `.DS_Store` — службові файли Finder у macOS.

4. Встановіть залежності — у корені репозиторію:

   ```
   npm install
   ```

   npm може повідомити про вразливості рівня moderate в залежностях ESLint. Вони стосуються лише інструментів розробки й не потрапляють у збірку. **Не запускайте `npm audit fix --force`:** він «виправляє» вразливості, понижуючи версії пакетів. У цьому проєкті він замінив би `eslint-config-prettier` 10 на 6.15, а `@eslint/js` 10 — на 9, і жодної вразливості не виправив би.

5. Зробіть перший коміт:

   ```
   git add .
   git commit -m "chore: scaffold React app with Vite"
   ```

**Де що робити.** Відкривайте в редакторі теку `react-labs` — корінь репозиторію: там лежать налаштування редактора й `AGENTS.md`. Команди `npm run …` виконуйте в терміналі в теці `lab-1/` (`cd lab-1`). Далі шляхи до файлів проєкту (`src/App.jsx`, `package.json`) — відносно `lab-1/`, а про файли в корені сказано «у корені репозиторію».

## 4.3. Структура проєкту й скрипти

| Файл чи тека | Що це |
|---|---|
| `index.html` | сторінка, з якої починає Vite: `<div id="root">` і `<script type="module" src="/src/main.jsx">` |
| `src/main.jsx` | точка входу: `createRoot(...).render(...)` малює `<App />` всередині `<StrictMode>` |
| `src/App.jsx` | кореневий компонент; у шаблоні — лічильник на `useState` |
| `src/App.css`, `src/index.css` | стилі |
| `src/assets/` | файли, які імпортують з коду; у збірку потрапляють з хешем у назві |
| `public/` | файли, які копіюються в збірку як є |
| `vite.config.js` | налаштування Vite з плагіном `@vitejs/plugin-react` |
| `eslint.config.js` | налаштування ESLint у форматі flat config |
| `package.json` | залежності й скрипти лабораторної; точні версії фіксує кореневий `package-lock.json` |

Команди виконуйте в теці `lab-1/`:

| Команда | Що робить |
|---|---|
| `npm run dev` | dev-сервер на http://localhost:5173 з HMR |
| `npm run build` | production-збірка в теку `dist/` |
| `npm run preview` | локальний перегляд зібраного `dist/` на http://localhost:4173 |
| `npm run lint` | ESLint для всього проєкту |

1. Запустіть `npm run dev` і відкрийте http://localhost:5173. Відкрийте DevTools → ⚛ **Components**: виберіть `App` і знайдіть його стан у розділі **hooks**. Натисніть кнопку лічильника кілька разів і подивіться, як змінюється стан. Кнопкою вибору елемента у вкладці Components знайдіть компонент за елементом сторінки.
2. Зупиніть сервер (`Ctrl+C`). Виконайте `npm run build`: Vite виведе список файлів `dist/` з розмірами. Запустіть `npm run preview` і відкрийте http://localhost:4173.
3. Теки `node_modules/` (у корені репозиторію) і `dist/` є в `.gitignore`: їх не комітять, бо npm і Vite відтворюють їх з `package-lock.json` і з коду.

## 4.4. Prettier

1. Встановіть Prettier і `eslint-config-prettier`. Prettier радить фіксувати точну версію (`--save-exact`): навіть patch-реліз може трохи змінити форматування, і в учасників команди воно розійдеться.

   ```
   npm install --save-dev --save-exact prettier
   npm install --save-dev eslint-config-prettier
   ```

   Команди виконуйте в теці `lab-1/`: npm додасть пакети в `lab-1/package.json`, а їхні версії запише в кореневий `package-lock.json`.
2. Створіть у `lab-1/` файл `.prettierrc.json`:

   ```
   {
     "semi": false,
     "singleQuote": true
   }
   ```

   Це стиль шаблону Vite: без крапок з комою, з одинарними лапками. Тож Prettier не змінить у файлах шаблону жодного рядка. Решта параметрів — типові: ширина рядка 80, відступ 2 пробіли. `node_modules/` і файли з `.gitignore` теки (`dist/`) Prettier 3 пропускає сам, тож окремий `.prettierignore` не потрібен.
3. Додайте в `package.json` до `"scripts"` два рядки:

   ```
   "format": "prettier . --write",
   "format:check": "prettier . --check"
   ```

4. Підключіть `eslint-config-prettier` в `eslint.config.js` — імпортом і останнім елементом масиву:

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import eslintConfigPrettier from 'eslint-config-prettier/flat'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
  eslintConfigPrettier,
])
```

   `eslint-config-prettier` вимикає правила ESLint, які сперечаються з Prettier про форматування. Він стоїть останнім, бо пізніші елементи масиву перекривають ранніші. У поточних наборах правил таких немає, тож зараз він нічого не змінює — і захищає від конфліктів, коли ви додасте плагін зі стильовими правилами. Перевірити це можна командою, яку пакет має для цього:

```
npx eslint-config-prettier src/main.jsx
```

5. Запустіть `npm run lint` і `npm run format:check`: обидві команди мають пройти без помилок. Вони перевіряють лише файли `lab-1/`. Зробіть коміт `chore: add Prettier`.

## 4.5. Редактор

**VS Code, Cursor, Windsurf.** Створіть у корені репозиторію (не в `lab-1/`) теку `.vscode/` з двома файлами: вони діятимуть для всіх лабораторних. `extensions.json` пропонує розширення кожному, хто відкриє репозиторій:

```
{
  "recommendations": ["dbaeumer.vscode-eslint", "esbenp.prettier-vscode"]
}
```

`settings.json` вмикає Prettier під час збереження файлу й виправлення ESLint, які можна застосувати автоматично:

```
{
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "eslint.workingDirectories": [{ "pattern": "./lab-*/" }]
}
```

Останній рядок каже розширенню ESLint, що кожна тека `lab-*` — окремий проєкт зі своїм `eslint.config.js`.

`.gitignore` шаблону ігнорує `.vscode/` всередині `lab-1/`, а на кореневу теку не діє, тож обидва файли потраплять у Git. Перевірте `git status -u`: серед нових мають бути `.vscode/extensions.json` і `.vscode/settings.json`. Без `-u` Git показує нову теку одним рядком `.vscode/`, і не видно, які файли в ній.

**WebStorm.** У **Settings** → **Languages & Frameworks** → **JavaScript** → **Code Quality Tools** → **ESLint** виберіть **Automatic ESLint configuration** і позначте **Run eslint --fix on save**. У **Settings** → **Languages & Frameworks** → **JavaScript** → **Prettier** виберіть **Automatic Prettier configuration** і позначте **Run on save**. Налаштування WebStorm живуть у `.idea/`, яку ігнорує кореневий `.gitignore`, тож у звіт додайте знімок екрана цих двох сторінок.

Зробіть коміт `chore: share editor settings`.

## 4.6. Перевірка інструментів

Ці перевірки нічого не залишають у проєкті: після кожної поверніть файли до останнього коміту (`git status` має бути чистим).

**П1. Fast Refresh.** Запустіть `npm run dev`, натисніть лічильник до 5. У `src/App.jsx` замініть текст `Get started` своїм ім'ям і збережіть файл. Сторінка оновиться без

перезавантаження, а лічильник залишиться на 5: React Fast Refresh замінив код компонента й зберіг його стан. Поверніть текст.

**П2. ESLint.** Створіть тимчасовий файл `src/LintDemo.jsx`:

```
import { useEffect, useState } from 'react'

export default function LintDemo({ items }) {
  const [total, setTotal] = useState(0)
  const label = 'Разом'

  if (items.length === 0) {
    const [message] = useState('Порожньо')
    return <p>{message}</p>
  }

  useEffect(() => {
    setTotal(items.length)
  }, [items])

  return <p>{total}</p>
}

export function formatTotal(total) {
  return `Разом: ${total}`
}
```

Редактор підкреслить помилки ще до запуску лінтера. Запустіть `npm run lint`: має бути 5 помилок від 4 правил. Зробіть знімок екрана для звіту й запишіть, що означає кожне правило. Видаліть `src/LintDemo.jsx`.

**П3. Prettier.** У `src/App.jsx` замініть `import './App.css'` на `import "./App.css";` і збережіть **без форматування**: у VS Code — **File: Save without Formatting** (`Ctrl+K S`), у WebStorm — тимчасово зніміть **Run on save**. Запустіть `npm run format:check`: Prettier назве `src/App.jsx`. Тепер збережіть файл звичайно (`Ctrl+S`): лапки й крапка з комою зміняться, а `npm run format:check` пройде.

**П4. Dev проти збірки.** Відкрийте DevTools → **Network**, фільтр **JS**, і перезавантажте сторінку спершу на `npm run dev` (http://localhost:5173), потім на `npm run preview` після `npm run build` (http://localhost:4173). Порівняйте, скільки JS-файлів завантажує сторінка в кожному режимі і як вони називаються.

## 4.7. AGENTS.md для React-проєкту

`AGENTS.md` — звичайний Markdown-файл у корені репозиторію з інструкціями для AI-агента: команди перевірки, структура проєкту, правила коду. Агент читає його на

початку кожної сесії, тож правила не треба повторювати в кожному запиті.

Для React такий файл особливо корисний. Агенти навчалися на великій кількості старого React-коду й часто пропонують те, чого в сучасному React уже немає або що не радять: класові компоненти, `ReactDOM.render`, Create React App, `propTypes`, зайві `useEffect`. `AGENTS.md` прямо називає версії й правила вашого проєкту.

**Які агенти читають `AGENTS.md`:**

| Агент | Як підхоплює `AGENTS.md` |
|---|---|
| GitHub Copilot у VS Code | автоматично з кореня робочої теки; керує налаштування `chat.useAgentsMdFile`, увімкнене за замовчуванням |
| Cursor, Windsurf, Codex, Junie (JetBrains) | автоматично |
| Claude Code | автоматично, якщо в проєкті немає `CLAUDE.md`; якщо є — Claude Code читає лише `CLAUDE.md`, тож додайте в нього рядок `@AGENTS.md` |
| Gemini CLI | після налаштування: у `.gemini/settings.json` — `{ "context": { "fileName": ["AGENTS.md", "GEMINI.md"] } }` |

**Як писати `AGENTS.md`:**

- Пишіть те, чого агент не дізнається з коду сам: команди перевірки, версії, заборони, рішення, ухвалені в проєкті.
- Одне правило — одне конкретне речення: «Run `npm run lint`», а не «пиши якісний код».
- Те, що може перевірити інструмент, хай перевіряє інструмент. `AGENTS.md` не переписує правила ESLint, а вимагає запустити `npm run lint` і не вимикати правил.
- Файл росте разом із проєктом: коли агент удруге робить ту саму помилку, допишіть правило.
- Жодних секретів: файл лежить у публічному репозиторії.

1. Створіть у корені репозиторію `AGENTS.md` — один на всі лабораторні. Почніть із шаблону й допишіть власні правила, якщо вони у вас є. Шаблон англійською: так його однаково розуміють усі агенти, а українська залишається для текстів інтерфейсу.

Розділ **Scope** тримає агента в теці лабораторної, про яку йдеться в запиті: коли лабораторних стане кілька, він не правитиме сусідні.

```
# AGENTS.md

Labs for the React course, one folder per lab. Each `lab-N/` folder is a
separate single-page app on React 19 and Vite 8, in JavaScript with JSX. No
TypeScript yet. User-facing text is in Ukrainian.

## Repository layout

- `lab-N/`: one lab, with its own `package.json`, ESLint and Prettier
  configuration
- `package.json` in the root: npm workspaces over the `lab-*` folders, so
all
  labs share one `package-lock.json` and one `node_modules/`
- `AGENTS.md`, `.gitattributes`, `.gitignore`, `.vscode/`: shared by all
labs

## Commands

Run `npm install` in the repository root. Run every other command inside
the
lab's folder, for example `lab-1/`:

- `npm run dev`: dev server at http://localhost:5173
- `npm run lint`: ESLint
- `npm run format`: format the lab's files with Prettier
- `npm run format:check`: check formatting without changing files
- `npm run build`: production build into the lab's `dist/`

A task is done only when `npm run lint`, `npm run format:check` and
`npm run build` pass in the lab's folder. Run all three and report their
results.

## Scope

- Work only in the folder of the lab the task names. If the task doesn't
name
  a lab, ask.
- Don't change other labs' folders, and don't import code from one lab into
  another.

## Lab structure

- `index.html`: the page Vite serves; the app mounts into `<div id="root">`
- `src/main.jsx`: entry point; renders `<App />` inside `<StrictMode>`
```

```
- `src/App.jsx`: root component
- `src/components/`: one component per file, named in PascalCase
  (`ProfileCard.jsx`)
- `public/`: static files, copied to the build as is

## React rules

- Function components and hooks only. No class components.
- Use current React 19 APIs: `createRoot`, not `ReactDOM.render`. No
  `propTypes`, and no `defaultProps` on function components (use default
  parameter values instead).
- Follow the Rules of Hooks: call hooks only at the top level of a
component
  or a custom hook, never in a condition, a loop or after an early return.
- Keep rendering pure. Compute values from props and state during render
  instead of copying them into state with `useEffect`.
- Give list items a stable `key` from the data, never the array index.
- Use semantic HTML: `<button>` for actions, `<a href>` for navigation, a
  `<label>` for every form field, `alt` on every `<img>`.
- Don't use `dangerouslySetInnerHTML`.

## Dependencies and configuration

- Don't add, remove or upgrade npm packages without asking first, and say
why
  a package is needed. Install a package from inside the folder of the lab
  that needs it, never in the repository root.
- Don't run `npm audit fix --force`.
- Don't change the root `package.json`, or a lab's `eslint.config.js`,
  `.prettierrc.json`, `vite.config.js` or `package.json` scripts, unless
the
  task is about them.
- Never turn off ESLint rules or add `eslint-disable` comments. Fix the
cause.

## How to work

- Before editing, read the relevant files and list the files you will
change.
- Keep each change small and limited to the task.
- After editing, summarise what changed and show the check results.
- Never put secrets (API keys, tokens, passwords) in the code. Every
  `VITE_*` variable ends up in the public bundle.
```

2. `npm run format:check` у `lab-1/` кореневих файлів не перевіряє. Збережіть `AGENTS.md` у редакторі — Prettier відформатує його під час збереження — і перевірте кореневі

файли з кореня репозиторію:

```
npx prettier --check AGENTS.md .vscode
```

У WebStorm так не вийде: Prettier за замовчуванням не форматує там Markdown під час збереження, а теки `.vscode/` немає, тож команда завершилася б помилкою `No files matching the pattern were found`. Відформатуйте файл командою `npx prettier --write AGENTS.md`.
Зробіть коміт `docs: add AGENTS.md`.

3. **Запит 1 — чи прочитав агент файл.** Почніть нову сесію агента в режимі без редагування (Ask чи Plan) і надішліть:

> Не змінюй жодного файлу. Коротко перекажи правила цього репозиторію: як у ньому розкладено лабораторні, якими командами й у якій теці ти перевірятимеш свою роботу, яких правил React маєш дотримуватися і що тобі заборонено робити без мого дозволу.

Агент має назвати команди й правила саме з вашого `AGENTS.md`. Якщо відповідь загальна, без ваших команд, — агент файлу не бачить: перевірте таблицю вище й налаштування агента.

4. **Запит 2 — зміна за правилами.** Перемкніть агента в режим Agent і надішліть, підставивши свої дані:

> У лабораторній `lab-1` заміни демонстраційний вміст `src/App.jsx` на картку «Про мене». Ім'я: <ваше ім'я>. Група: <ваша група>. Технології, які я хочу опанувати в цьому курсі: <три технології>. На картці — ім'я, група, список технологій і кнопка «Подобається» з лічильником натискань. Картка — окремий компонент `ProfileCard`, дані він отримує через props. Видали стилі й зображення шаблону, які стануть непотрібними. Спочатку покажи план і список файлів, які зміниш.

5. **Перевірте роботу агента** — за `git diff`, у терміналі й у браузері:

| Що перевірити | Правило `AGENTS.md` |
|---|---|
| план до змін | агент перелічив файли, перш ніж редагувати |
| межі | змінено лише файли в `lab-1/` |
| розташування | компонент лежить у `lab-1/src/components/ProfileCard.jsx` |
| React | функційний компонент, стан лічильника — `useState`, `key` у списку не з індексу |
| семантика | лічильник — `<button>`, а не `<div onClick>` |
| залежності й конфіг | кореневий `package.json` не змінено, а в `lab-1/` — `package.json`, `eslint.config.js`, `.prettierrc.json`, `vite.config.js` |

| Що перевірити | Правило `AGENTS.md` |
|---|---|
| перевірки | агент запустив у `lab-1/` `npm run lint`, `npm run format:check` і `npm run build` і показав результат |

Відкрийте сторінку й у ⚛ **Components** знайдіть `ProfileCard`: його props і стан лічильника. Закомітьте зміну агента без виправлень: `feat: add profile card`. Якщо агент порушив правило, виправте це власноруч окремим комітом, наприклад `fix: use button for the like counter`, і подумайте, яке речення в `AGENTS.md` запобігло б цьому наступного разу.

## 4.8. GitHub і Vercel

1. Створіть на GitHub порожній репозиторій `react-labs` — без README, `.gitignore` і ліцензії: вони вже є в проєкті або не потрібні.
2. Підключіть його й відправте код:

```
git remote add origin https://github.com/<username>/react-labs.git
git push -u origin main
```

3. Перевірте репозиторій у браузері. У корені — `AGENTS.md`, `.gitattributes`, `.gitignore`, `package.json`, `package-lock.json` і обидва файли `.vscode/` (`extensions.json` і `settings.json`); у `lab-1/` — проєкт з `.prettierrc.json`. Тек `node_modules/` і `dist/` немає. Історія комітів:
   1. `chore: scaffold React app with Vite` — шаблон без змін у `lab-1/` і кореневі `package.json`, `package-lock.json`, `.gitattributes` і `.gitignore`;
   2. `chore: add Prettier`;
   3. `chore: share editor settings`;
   4. `docs: add AGENTS.md`;
   5. `feat: add profile card` — зміна агента без виправлень;
   6. виправлення, якщо агент порушив правила `AGENTS.md`.
4. Увійдіть на vercel.com → **Add New** → **Project** і імпортуйте репозиторій `react-labs`. Перед **Deploy** налаштуйте:

| Поле | Значення |
|---|---|
| Project Name | `react-lab-1` |
| Root Directory | **Edit** → `lab-1` |
| Framework Preset | Vite (Vercel розпізнає його сам) |
| Output Directory | `dist` |

Кожна лабораторна — окремий проєкт Vercel з того самого репозиторію: `react-lab-2` з Root Directory `lab-2` і так далі. Тому назва проєкту — за лабораторною, а не за репозиторієм. Vercel встановить залежності з кореневого `package-lock.json`, виконає `npm run build` у `lab-1/` і опублікує `lab-1/dist/`. Тому `dist/` і не комітять.

5. Відкрийте адресу `https://<project>.vercel.app/`: має відкритися картка «Про мене». Кожен наступний `git push` у `main` створює новий деплой — перевірте це на будь-якій наступній зміні. Коли лабораторних стане кілька, Vercel деплоїтиме лише ті, чиї теки змінив коміт: у репозиторії з npm workspaces нові проєкти пропускають незмінені лабораторні за замовчуванням (перемикач **Skip deployment** у блоці **Root Directory**, **Settings** → **Build and Deployment**). Винятком є файли в корені репозиторію (`AGENTS.md`, `.vscode/`, кореневий `package.json`): вони не належать жодній лабораторній, тож після їхньої зміни Vercel перезбирає всі лабораторні.

## 4.9. Якщо ЛР-1 уже виконано в репозиторії `react-lab-1`

Проєкт не треба створювати заново: його переносять у теку `lab-1/`, а репозиторій перейменовують на `react-labs`. Історія комітів зберігається. Перед початком зупиніть dev-сервер і закомітьте всі зміни (`git status` має бути чистим).

1. **GitHub.** У репозиторії відкрийте **Settings** → **General**, у полі **Repository name** введіть `react-labs` і натисніть **Rename**. Старі посилання, зокрема у вже надісланому звіті, GitHub перенаправляє на нову адресу. Оновіть адресу й у локальному репозиторії:

   ```
   git remote set-url origin https://github.com/<username>/react-labs.git
   ```

2. **Перенесення.** У корені проєкту виконайте:

   ```
   git rm package-lock.json
   mkdir lab-1
   git mv .gitignore .prettierrc.json README.md eslint.config.js index.html
   package.json vite.config.js public src lab-1
   ```

   `package-lock.json` видаляють, бо спільний lock-файл npm створить у корені. `.gitattributes`, `.vscode/` і `AGENTS.md` залишаються в корені. Якщо якогось файлу з переліку у вас немає, `git mv` зупиниться з помилкою `bad source` і нічого не перенесе: приберіть цей файл з команди й повторіть її.
   Видаліть з кореня теки `node_modules/` і `dist/` — у Git їх немає, npm і Vite відтворять їх: у провіднику редактора, у PowerShell командою `Remove-Item -Recurse -Force node_modules, dist` або в bash — `rm -rf node_modules dist`.

3. **Кореневі файли.** Створіть у корені `package.json` і `.gitignore` з §4.2, крок 3. У VS Code, Cursor і Windsurf замініть вміст `.vscode/settings.json` файлом з §4.5: від попереднього він відрізняється лише рядком `"eslint.workingDirectories"`. Замініть

`AGENTS.md` шаблоном з §4.7, перенісши в нього власні правила. Рядок `!.vscode/settings.json` у `lab-1/.gitignore` тепер нічого не робить, його можна видалити.

4. **Перевірка.** У корені виконайте `npm install` і `npx prettier --check AGENTS.md .vscode` (у WebStorm — `npx prettier --write AGENTS.md`, як у §4.7), потім у `lab-1/` — `npm run lint`, `npm run format:check` і `npm run build`. Усі команди мають пройти без помилок.
5. **Коміт.** У корені:

   ```
   git add .
   git status
   git commit -m "chore: move lab 1 into lab-1/"
   ```

   `git status` має показати перенесені файли як перейменовані (`renamed`): так Git зберігає їхню історію, її видно в `git log --follow lab-1/src/App.jsx`. Виняток — `package.json` і `.gitignore`: у корені їх замінили нові файли, тож Git покаже їх як змінені (`modified`) у корені й нові (`new file`) у `lab-1/`. `git log --follow` знаходить історію й для них.
6. **Vercel.** Відкрийте проєкт `react-lab-1` → **Settings**:
   - **Git**: підключеним має бути `react-labs`. Якщо Vercel показує помилку або стару назву, натисніть **Disconnect** і підключіть `react-labs` знову;
   - **Build and Deployment** → **Root Directory**: введіть `lab-1`, увімкніть **Skip deployment** і натисніть **Save**.

   Потім виконайте `git push` і перевірте, що новий деплой зібрано й адреса `https://<project>.vercel.app/` показує картку «Про мене». Якщо деплой завершився помилкою, перевірте Root Directory і натисніть **Redeploy** на останньому деплої.

Якщо звіт (§5) ще не надіслано, вкажіть у ньому нову адресу репозиторію. До історії комітів з §4.8 додається ще один коміт: `chore: move lab 1 into lab-1/`.

# 5. Вимоги до звіту

**Тема листа:**

```
[React] ЛР-1: Налаштування середовища розробки
```

У листі вкажіть репозиторій і адресу застосунку на Vercel:

```
GitHub repository: https://github.com/<username>/react-labs
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. результат `node --version`, `npm --version` і `git --version`; редактор і встановлені розширення (для WebStorm — знімки екрана налаштувань з §4.5);
2. знімок екрана застосунку з відкритою вкладкою ⚛ **Components**;
3. знімок екрана `npm run lint` з помилками П2 і коротко — що означає кожне з 4 правил;
4. знімок екрана `npm run lint`, `npm run format:check` і `npm run build` без помилок після всіх комітів;
5. назву й версію AI-агента, модель, дату роботи; звідки агент бере правила проєкту;
6. короткий висновок: що налаштовано й перевірено, що було складним.

# 6. Усний захист

Формат здачі передбачає усну співбесіду за результатами роботи. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного проєкту. Викладач може попросити показати відповідь у коді, терміналі чи DevTools.

**Питання для підготовки:**

*Node.js і npm*

- Навіщо фронтенд-проєкту Node.js, якщо застосунок працює в браузері?
- Що таке LTS-версія Node.js? Звідки відомо, яку версію Node.js вимагає Vite чи ESLint?
- Що зберігається в `package.json`, а що — в `package-lock.json`? Чому lock-файл комітять, а `node_modules/` — ні?
- Чим `dependencies` відрізняються від `devDependencies`? Чому `react` — в одному, а `eslint` — в іншому?
- Що означають версії `^19.3.1` і `3.9.10` без `^`? Навіщо Prettier встановлюють з `--save-exact`?
- Що показує `npm audit`? Чому `npm audit fix --force` небезпечний?
- Що таке npm workspaces? Чому в репозиторії один `package-lock.json` і одна `node_modules/` на всі лабораторні, а `package.json` — у кожній?

*Vite*

- Що робить Vite у режимі dev, а що — під час `vite build`?

- Чому dev-сервер не збирає застосунок в один файл? Що ви побачили у вкладці Network у П4?
- Що таке HMR і React Fast Refresh? Чому в П1 лічильник не скинувся?
- Навіщо хеш у назвах файлів у `dist/assets/`?
- Чим `public/` відрізняється від `src/assets/`?
- Чому `index.html` лежить у корені проєкту, а не в `public/`?
- Що робить `@vitejs/plugin-react`?

*React-проєкт*

- Що роблять `createRoot` і `render` у `src/main.jsx`?
- Навіщо `<StrictMode>`? Що він змінює в режимі dev і чи діє в production?
- Що таке JSX і хто перетворює його на JavaScript?
- Що показує вкладка Components у React DevTools? Як знайти компонент за елементом на сторінці?

*ESLint і Prettier*

- Чим лінтер відрізняється від форматера? Наведіть помилку, яку знайде ESLint, але не Prettier.
- Що таке flat config? З яких частин складається ваш `eslint.config.js`?
- Що перевіряють правила `eslint-plugin-react-hooks` з П2? Чому хук не можна викликати в умові чи після раннього `return`?
- Чому `react-refresh/only-export-components` не дозволяє експортувати функцію поруч із компонентом?
- Навіщо `eslint-config-prettier` і чому він стоїть останнім?
- Чим Oxlint відрізняється від ESLint? Чому в курсі обрано ESLint?
- Навіщо комітити `.prettierrc.json` і `.vscode/settings.json`, якщо кожен може налаштувати свій редактор?
- Що робить `.gitattributes`? Що показав би `npm run format:check` у свіжому клоні на Windows без нього?

*Деплой на Vercel*

- Що запускає новий деплой на Vercel? Що Vercel робить з репозиторієм: яку команду виконує і яку теку публікує?
- Чому для деплою не потрібно комітити `dist/`?
- Чим застосунок на Vercel відрізняється від `npm run preview`, а чим — від `npm run dev`?
- Як розгорнути кілька лабораторних з одного репозиторію? Що задає Root Directory і чому коміт у `lab-2/` не перезбирає `react-lab-1`, а зміна `AGENTS.md` перезбирає?

*AI-агент і AGENTS.md*

- Що таке `AGENTS.md` і коли агент його читає? Як ви перевірили, що ваш агент його прочитав?
- Чим правило в `AGENTS.md` відрізняється від того самого речення в запиті?
- Чому правило «запусти `npm run lint`» надійніше за правило «не порушуй правил хуків»?
- Які застарілі підходи React агенти пропонують найчастіше і чому?
- Чи дотримався агент усіх правил `AGENTS.md` у запиті 2? Як ви це перевірили?
- Чому значення змінних `VITE_*` не можна вважати секретами?

# 7. Критерії готовності

Роботу зараховано після захисту (§6); вона оцінюється в 1 бал.

| Критерій | Мінімальна вимога |
|---|---|
| Node.js | Node.js 22.13 або новіший; `node`, `npm` і `git` працюють |
| Репозиторій | `react-labs` з npm workspaces у кореневому `package.json`; ЛР-1 — у теці `lab-1/` |
| Проєкт | шаблон Vite «React + JavaScript» з ESLint; `npm run dev`, `npm run build` і `npm run preview` працюють |
| ESLint | `npm run lint` без помилок; редактор підкреслює помилки |
| Prettier | `.prettierrc.json`, скрипти `format` і `format:check`, `eslint-config-prettier`; `npm run format:check` без помилок |
| Редактор | форматування під час збереження; `.vscode/extensions.json` і `.vscode/settings.json` у корені репозиторію або знімки налаштувань WebStorm |
| React DevTools | встановлено; видно дерево компонентів, props і стан |
| AGENTS.md | у корені репозиторію; агент його читає; зміну за запитом 2 перевірено й закомічено |
| GitHub | коміти §4.8 (і коміт перенесення з §4.9, якщо був); немає `node_modules/` і `dist/`; є `.gitattributes` |
| Vercel | проєкт з Root Directory `lab-1`; `https://<project>.vercel.app/` відкриває картку «Про мене» з останнього коміту `main` |
| Звіт | усі пункти §5 |

# 8. Список джерел


1. React. Build a React app from Scratch
2. React. Editor Setup
3. React. React Developer Tools

4. React. `<StrictMode>`
5. React. Rules of Hooks
6. React. eslint-plugin-react-hooks
7. React. You Might Not Need an Effect
8. Vite. Getting Started
9. Vite. Features
10. Vite. Static Asset Handling
11. Vite. Building for Production
12. Vite. Env Variables and Modes
13. Node.js. Download Node.js
14. Node.js. Node.js Releases
15. npm. package.json
16. npm. package-lock.json
17. npm. npm audit
18. npm. Workspaces
19. ESLint. Configuration Files
20. Oxc. Oxlint
21. Prettier. Install
22. Prettier. Options
23. Prettier. Integrating with Linters
24. Prettier. Ignoring Code
25. Prettier. eslint-config-prettier
26. Visual Studio Marketplace. ESLint
27. Visual Studio Marketplace. Prettier - Code formatter
28. JetBrains. ESLint in WebStorm
29. JetBrains. Prettier in WebStorm
30. Git. gitattributes
31. GitHub. Configuring Git to handle line endings
32. GitHub. Renaming a repository
33. Vercel. Vite on Vercel
34. Vercel. Deploying Git Repositories with Vercel
35. Vercel. Using Monorepos
36. AGENTS.md. A simple, open format for guiding coding agents
37. Visual Studio Code. Custom instructions in VS Code
38. Cursor. Rules
39. Claude Code. How Claude remembers your project

40. Gemini CLI. Provide context with GEMINI.md files